

Class Project Phase II Submission Template

Group 6

Contributors: Leheng Chen, Alexa Quijano

1. Document explaining and justifying design patterns you have identified for this project. A minimum of three design patterns are required.

Design Pattern 1: Factory Pattern

Explanation as to how this design pattern improves your design (Min 200 words):

Purpose

The Factory Pattern provides a centralized way to create objects without requiring other classes to know exactly how those objects are created. This helps reduce dependencies between classes and makes the system easier to maintain.

Usage

In the SunDevil Connect system, the Factory Pattern can be used to create users and events. Based on our Phase I design, the system includes User, Student, ClubLeader, Administrator, Club, and Event classes. A UserFactory class could create Student, ClubLeader, or Administrator objects depending on the role of the user who logs in. An EventFactory class could create Event objects when a club leader creates a new event through the system. Instead of controllers creating objects directly, classes such as AuthController, ClubController, and EventController could request objects from the factory. This keeps the object creation process organized and in one location.

Benefits

The Factory Pattern improves the design because it keeps object creation separate from the rest of the application. This makes the code easier to read and easier to maintain. Controllers do not need to know how specific objects are created because the factory handles that responsibility. If new user roles or event categories are added in the future, changes can be made in the factory instead of throughout the entire system. This reduces code duplication and makes future updates simpler. The pattern also improves flexibility because different types of objects can be created without changing the client code. Since SunDevil Connect is expected to manage many users, clubs, and events, having a centralized object creation process helps keep the system organized and scalable. Overall, the Factory Pattern creates a cleaner design and makes the application easier to expand as new features are added.

Design Pattern 2: Observer Pattern

Explanation as to how this design pattern improves your design (Min 200 words):

Purpose

The Observer Pattern creates a one-to-many relationship between objects. When one object changes, all related objects are automatically notified. This helps keep information updated throughout the system without requiring users to constantly check for changes.

Usage

In the SunDevil Connect system, the Observer Pattern can be used to improve communication between clubs and students. Based on our Phase I design, students can join clubs, register for events, and receive updates from organizations they are interested in. The Club class can act as the subject, while Student objects act as observers. When a club leader creates a new event, updates event information, cancels an event, or posts an announcement, the Club object can notify all subscribed students. This allows students to stay informed about activities happening within clubs they follow. The notifications could be displayed in the application and could later be expanded to support email or mobile notifications.

Benefits

The Observer Pattern improves the design because it helps keep students informed without requiring them to manually check club pages for updates. It also reduces dependencies between classes because the Club object only needs to send notifications to its observers and does not need to know how those notifications are delivered. This makes the system easier to maintain and modify. If new notification methods are added in the future, such as text messages or push notifications, they can be added without changing the Club class itself. This pattern also supports one of the main goals of SunDevil Connect, which is increasing student engagement in campus activities. Students can quickly learn about new events, announcements, and club updates, helping them stay involved and connected. Overall, the Observer Pattern improves communication, flexibility, and the user experience of the system.

Design Pattern 3: Singleton Pattern

Explanation as to how this design pattern improves your design (Min 200 words):

Purpose

The Singleton Pattern ensures that only one instance of a class exists throughout the application. It also provides a single access point that can be used by other classes whenever that functionality is needed. This pattern is commonly used for services that need centralized control.

Usage

In the SunDevil Connect system, the Singleton Pattern can be applied to authentication and login management. During Phase I, our design included Login, Display Login Screen, and Verify User Credentials use cases. We also identified the AuthController class as part of the system. A Singleton AuthenticationManager class could be responsible for validating user credentials, managing login sessions, and determining access permissions. All Students, ClubLeaders, and Administrators would use the same authentication instance when logging into the system.

Controllers such as AuthController, ClubController, EventController, and AdminController could access this shared authentication service whenever user verification is required.

Benefits

The Singleton Pattern improves the design by providing one central location for authentication. This helps ensure that every user is validated using the same rules and security checks. If multiple authentication objects were created, different parts of the system could potentially handle login requests differently, which could lead to inconsistencies. Using a Singleton prevents this problem by making sure there is only one authentication manager. Another benefit is that it improves resource management because the system only creates one authentication object instead of several copies. This reduces unnecessary memory usage and simplifies maintenance. If authentication rules need to be updated in the future, the changes only need to be made in one place. Since authentication affects every user of the application, having a centralized and consistent solution is important. Overall, the Singleton Pattern improves security, consistency, maintainability, and organization throughout the SunDevil Connect system.

Note: If you have additional design patterns identified, please list them as well using the same format above

Design Pattern 4: Strategy Pattern

Explanation as to how this design pattern improves your design (Min 200 words):

Purpose:

The Strategy Pattern defines a family of algorithms, encapsulates each one into a separate class, and makes them interchangeable. This pattern lets the algorithm vary independently from the clients that use it. It is particularly useful for replacing complex conditional statements (like multiple if-else blocks) with delegating tasks to distinct strategy objects.

Usage:

In the SunDevil Connect system, the Strategy Pattern can be perfectly applied to the "Filter Events" use case. Based on Phase I, students can narrow down event results by various criteria such as date, location, category, popularity, and free/paid status. Instead of writing a massive block of conditional logic inside the EventController to handle every possible filtering combination, we can create an **EventFilterStrategy** interface.

This interface will declare a filter(events) method. We then create concrete classes that implement this interface, such as DateFilterStrategy, CategoryFilterStrategy, and CostFilterStrategy, connecting them to EventFilterStrategy with dashed lines ending in hollow triangles. When a student selects a filter on the EventSearchPage, the EventController simply delegates the filtering task to the corresponding strategy object.

Benefits:

The Strategy Pattern significantly improves the design by adhering to the Open/Closed Principle. If the university decides to add new filtering methods in the future, developers can simply create a new concrete strategy class without touching the existing EventController code. This reduces the risk of introducing bugs into the core application logic. Furthermore, it keeps the EventController clean and focused on coordinating data between the UI and entities, rather than being bloated with heavy data-processing algorithms. This modular approach also makes unit testing much easier, as each filtering logic can be tested independently. Overall, it makes the system's event search functionality highly flexible, maintainable, and scalable.

2. Refined class diagram clearly shows the design pattern related to classes and other classes. Astah must be used to develop the class diagram. (Copy and paste your class diagram here. Make sure to use the correct UML notations. Points will be given for the completeness and the correctness of the diagram)

